

总体刚度矩阵组装与桁架结构求解程序

1. 文件结构

```
global_stiffness_homework/
├─ src/
│   ├── main.py          # 主程序入口
│   ├── model.py         # JSON 读取、自由度编号、LM 生成
│   ├── element.py       # 单元几何、刚度矩阵、应力和轴力
│   ├── assembly.py      # 总体刚度矩阵直接组装
│   ├── solver.py        # 缩减法处理位移边界条件并求反力
│   └─ postprocess.py    # 后处理和输出格式化
├─ examples/
│   ├── example_1d_bar.json
│   ├── example_2d_truss.json
│   └─ example_3d_truss_bonus.json
├─ results/
│   ├── example_1d_bar_output.txt
│   ├── example_2d_truss_output.txt
│   └─ example_3d_truss_bonus_output.txt
├─ report/
│   └─ global_stiffness_report.pdf
└─ run_all_examples.py
```

2. 运行环境

- Python 3.9 或以上
- NumPy

安装依赖：

```
pip install numpy
```

3. 运行方法

在压缩包根目录下运行全部算例：

```
python run_all_examples.py
```

单独运行某个算例：

```
cd src
python main.py ../examples/example_1d_bar.json --output
../results/example_1d_bar_output.txt
python main.py ../examples/example_2d_truss.json --output
../results/example_2d_truss_output.txt
```

4. 输入文件说明

输入文件采用 JSON 格式。节点号、单元连接 IEN、边界条件自由度号和载荷自由度号均采用从 1 开始的编号；程序内部自动转换为从 0 开始的数组下标。

二维桁架自由度顺序为：

$$d = [u1, v1, u2, v2, u3, v3]^T$$

三维桁架自由度顺序为：

$$d = [u1, v1, w1, u2, v2, w2, \dots]^T$$

5. 已完成内容

1. 前处理：读取 JSON 模型数据，包括节点坐标、单元连接、材料参数、截面积、边界条件和载荷。
2. 单元分析：支持一维杆单元、二维桁架单元和三维桁架单元。
3. 对号矩阵 LM：根据 IEN 自动生成。
4. 直接组装：采用 $K[LM[a,e], LM[b,e]] += Ke[a,b]$ 。
5. 边界条件：采用缩减法处理已知位移自由度。
6. 后处理：输出单元长度、方向余弦、应力和轴力。
7. 矩阵性质检查：输出对称性、奇异性、稀疏性和对角元非负性。
8. 验证算例：完成一维两单元杆结构和二维两杆桁架结构。
9. 附加扩展：程序可处理三维桁架单元，并给出一个空间三杆桁架示例。

6. 验证结果摘要

- 算例 1：得到 $d1 = 0.000000$, $d2 = 0.100000$, $d3 = 0.150000$ ，节点 1 反力 $r1 = -10.000000$ ，大小为 10，与外载平衡。
- 算例 2：得到 $u3 = 38.284271$, $v3 = -10.000000$ ；单元 1 应力 -10.000000 ，单元 2 应力 14.142136 。